

Vite & Gourmand

Documentation Technique

Réflexions technologiques, Environnement, Architecture, MCD, Diagrammes UML, API, Sécurité, Déploiement

Version 1.4 — 2026

ECF Studi — TP Développeur Web et Web Mobile

Élément	Lien
Application	https://vitegourmand33.vercel.app
API Backend	https://vite-gourmand-back-chap.onrender.com
GitHub Front	https://github.com/NicolasVrignaudVG/Vite-Gourmand
GitHub Back	https://github.com/NicolasVrignaudVG/Vite-Gourmand-back

Journal des versions

Version	Date	Modifications
1.3	2026	Version initiale complète (architecture, MCD, API, sécurité, déploiement)
1.4	Août 2026	Campagne de correctifs de sécurité et de robustesse : en-têtes de sécurité côté front (6.5), règle de repli sur les routes API (6.3), résilience des services externes (6.6), validation de disponibilité des menus (5.2), génération du numéro de commande (4.1), composant d'accès aux données optimisé (8.1). Mise à jour de l'environnement de travail (2) et correction du type des montants (4.1). Extraction de la logique d'hydratation des menus dans un service dédié (8.1), alimentation dynamique des filtres thème et régime côté front (8.2).

1. Réflexions initiales technologiques

L'énoncé n'impose aucune technologie hormis l'usage conjoint d'une base relationnelle et d'une base non relationnelle. Les choix suivants ont été faits en amont du développement, chacun étant motivé par un besoin du projet :

1.1 Back-end

- **Symfony 8 (PHP 8.4)** : framework PHP le plus utilisé en entreprise en France. Architecture MVC claire, sécurité intégrée (rôles, hachage des mots de passe), ORM Doctrine, et Symfony Flex pour l'installation automatisée des bundles. PHP 8.4 est la version stable recommandée par Symfony 8, supportée avec correctifs de sécurité jusqu'à fin 2026 ; c'est également la version fixée par l'image Docker de production (php:8.4-apache). Ce choix a été fait en début de projet, alors que PHP 8.5 venait de paraître et que le support des dépendances tierces, en particulier LexikJWTAuthenticationBundle pour l'authentification qui n'était pas encore établi. Retenir la version stable précédente évitait de dépendre de mises à jour de compatibilité pendant le développement. Depuis, la version 3.2.0 du bundle intègre PHP 8.5 à ses tests d'intégration continue ; une montée de version est envisageable, mais l'alignement entre l'environnement de développement et l'image de production reste le critère prioritaire.
- Doctrine ORM : manipulation de la base via des objets PHP plutôt que du SQL manuel ; les requêtes sont automatiquement paramétrées, ce qui protège des injections SQL.
- LexikJWTAuthenticationBundle : authentification JWT, token stocké en cookie HttpOnly plutôt qu'en localStorage (inaccessible au JavaScript, protection contre le vol par XSS).
- MySQL 8.0 (Clever Cloud) : SGBD relationnel le plus répandu de l'écosystème PHP/Symfony et proposé par Clever Cloud ; le moteur InnoDB apporte transactions ACID et clés étrangères, indispensables à l'intégrité des données de commande.
- MongoDB Atlas : stockage des statistiques de vente par menu : la structure en documents JSON est plus souple qu'un schéma relationnel pour des données d'agrégation, et répond à l'exigence d'une base non relationnelle du cahier des charges.

1.2 Front-end

- HTML5 / CSS3 / JavaScript ES6+ vanilla : pas de framework JavaScript, pour garder une architecture légère et démontrer la maîtrise de chaque aspect du code. Un routeur hash personnalisé (Router/router.js) assure la navigation SPA sans rechargement de page.
- Bootstrap 5 (CDN) : menu mobile (offcanvas) et mise en page responsive, pour ne pas recoder ces composants interactifs depuis zéro.
- Chart.js (CDN) : graphiques interactifs de l'espace admin (commandes et chiffre d'affaires par menu). Choisi plutôt que Plotly car nettement plus léger (~60 Ko contre ~3 Mo) et suffisant pour le besoin.

1.3 Hébergement

- Render (Docker) : le back-end est packagé avec toutes ses dépendances (PHP 8.4, Apache, extensions) dans un conteneur portable : l'environnement de production est identique au

développement. Offre gratuite compatible ; limite connue : cold start de 30 à 60 s après 15 minutes d'inactivité.

- Vercel : plateforme optimisée pour les sites statiques (CDN intégré, déploiement automatique à chaque push GitHub) ; l'offre gratuite suffit pour un projet de démonstration.

2. Configuration de l'environnement de travail

Outil	Rôle et justification
MySQL Server 8.0 (Windows)	SGBD local installé en service Windows : même moteur et même version qu'en production (Clever Cloud), ce qui évite les écarts de comportement SQL
MySQL Workbench	Administration de la base locale : tables, relations, exécution de requêtes SQL
Composer	Gestion des dépendances PHP : installation des bundles (LexikJWT, NelmioCors, Doctrine...) en une commande
Visual Studio Code	Éditeur principal, extensions PHP et JavaScript (autocomplétion, détection d'erreurs)
Git + GitHub	Versionnage : deux dépôts distincts (front / back), en cohérence avec l'architecture découplée
Symfony CLI	Serveur de développement et outillage Symfony en local
Doctrine (bin/console)	Commandes dbal:run-sql et doctrine:migrations pour interroger et faire évoluer le schéma sans dépendre d'un client MySQL externe
Docker	Conteneurisation du backend, image identique en local et sur Render

Le déploiement est entièrement automatisé : chaque push sur la branche main déclenche un nouveau déploiement sur Vercel (front-end) et Render (back-end via Docker), sans intervention manuelle. La procédure d'installation locale complète est décrite dans le README.md de chaque dépôt.

Ajout v1.4 Environnement local et variables d'environnement. Les clés JWT sont fournies à l'application sous forme de contenu encodé en base64, via les variables JWT_SECRET_KEY et JWT_PUBLIC_KEY (processeur %env(base64:...)% dans lexik_jwt_authentication.yaml), et non sous forme de chemin de fichier. Cette convention est identique en local et sur Render, ce qui garantit qu'une configuration validée en développement se comporte à l'identique en production. Le fichier .env.local, non versionné, doit donc contenir le contenu des clés encodé en base64 : la commande d'encodage est documentée dans le README du dépôt back.

3. Architecture de l'application

3.1 Vue d'ensemble

Vite & Gourmand est une application web full-stack à architecture découplée : le frontend (SPA JavaScript vanilla) et le backend (API REST Symfony) sont deux projets distincts, déployés séparément et communiquant via HTTP/JSON. L'authentification est assurée par des tokens JWT transportés dans des cookies HttpOnly. En production, un proxy Vercel réécrit les appels /api/* du frontend vers le backend Render, de sorte que le navigateur ne voit qu'un seul domaine (voir section 6.4).

Couche	Technologie	Hébergement
Frontend	HTML / CSS / JS Vanilla (SPA)	Vercel

Backend API	PHP 8.4 + Symfony 8	Render (Docker)
Base SQL	MySQL 8.0	Clever Cloud
Base NoSQL	MongoDB Atlas (M0)	MongoDB Cloud
E-mails	Brevo API	Brevo (SaaS)
Livraison	OpenRouteService API	ORS Cloud

3.2 Stack technique

Technologie	Rôle
PHP 8.4	Langage backend : typage strict, attributs natifs, enums
Symfony 8	Framework PHP : Routing, Security, Serializer, Mailer
Doctrine ORM	Mapping objet-relationnel : Migrations, Repositories
LexikJWTAuthenticationBundle	Authentification JWT (RS256, clés RSA 4096 bits)
NelmioApiDocBundle	Documentation OpenAPI / Swagger UI (/api/doc)
NelmioSecurityBundle	En-têtes de sécurité HTTP + CSP sur les réponses de l'API
En-têtes Vercel (vercel.json)	En-têtes de sécurité et CSP sur les réponses du front (voir §6.5)
Extension MongoDB PHP	Connexion à MongoDB Atlas pour les statistiques
JavaScript ES6+	Frontend vanilla : fetch API, modules, async/await, routeur hash maison
Bootstrap 5 (CDN)	Menu mobile (offcanvas) et grille responsive
Chart.js (CDN)	Graphiques de l'espace admin (léger : ~60 Ko vs ~3 Mo pour Plotly)
Docker	Conteneurisation du backend (PHP 8.4 + Apache)
Déploiement continu	Webhooks Git natifs Vercel & Render (push main → deploy)

4. Modèle Conceptuel de Données

4.1 Entités principales

La base relationnelle MySQL comprend 18 tables : 14 entités métier et 4 tables d'association (commande_menu, commande_plat, menu_plat, plat_allergene). Les principales entités et leurs attributs :

Table utilisateur	Type	Description
id	INT PK	Identifiant unique auto-incrémenté
email	VARCHAR(255)	Adresse e-mail unique : login
password	VARCHAR(255)	Mot de passe haché bcrypt (coût 12)
nom / prenom	VARCHAR(100)	Identité du client
telephone / adresse	VARCHAR	Coordonnées (nullable)
pseudonyme	VARCHAR(50)	Nom public affiché avec les avis (minimisation RGPD à défaut, « Client vérifié »)
actif	TINYINT(1)	Compte actif (1) ou désactivé (0)
role_id	INT FK	Référence vers la table role
created_at	DATETIME	Date de création du compte

Table commande	Type	Description
id	INT PK	Identifiant unique
numero_commande	VARCHAR(20)	Référence unique, préfixe VG- suivi de 8 caractères hexadécimaux issus de random_bytes(4) ex. VG2C5E13A9
utilisateur_id	INT FK	Client ayant passé la commande
date_prestation	DATETIME	Date et heure souhaitées
adresse/ville/cp_livraison	VARCHAR	Lieu de livraison
nombre_personnes	INT	Nombre total de personnes
prix_menu / prix_livraison	DOUBLE	Détails tarifaires
prix_total / remise	DOUBLE	Total TTC et remise appliquée
statut	VARCHAR(20)	Statut courant (voir machine à états)
pret_materiel	TINYINT(1)	Matériel prêté au client

Correction v1.4 Type des montants. Les colonnes tarifaires étaient documentées en DECIMAL ; elles sont en réalité déclarées en float côté entité Doctrine, donc générées en DOUBLE par les migrations. DECIMAL reste le type canonique pour des montants monétaires, car il évite les erreurs d'arrondi propres à la virgule flottante. Cet écart est assumé comme dette technique identifiée : les montants manipulés (quelques centaines d'euros, arrondis à deux décimales par calculerPrix) restent très en deçà du seuil où l'imprécision devient perceptible, et une migration vers DECIMAL impacterait les entités, les migrations et la sérialisation. Elle est inscrite aux perspectives d'évolution.

Les tables commande_menu et commande_plat sont des tables d'association porteuses d'attributs (nombre de personnes, prix, remise par menu), permettant les commandes multi-menus. La table suivi_commande historise chaque changement de statut. La table refresh_token gère la rotation des jetons de rafraîchissement.

4.2 Relations principales

Entité 1	Card.	Entité 2	Description
utilisateur	1-N	commande	Un utilisateur passe plusieurs commandes
utilisateur	N-1	role	Un utilisateur possède exactement un rôle
utilisateur	1-N	refresh_token	Rotation des jetons de rafraîchissement
menu	1-N	menu_image	Un menu a plusieurs images
menu	N-1	theme / regime	Thématique et régime (nullable)
menu	N-N	plat	Composition via menu_plat
commande	N-N	menu	Multi-menus via commande_menu
commande	1-N	suivi_commande	Historique des statuts
commande	1-N	commande_plat	Plats choisis
plat	N-N	allergene	Allergènes via plat_allergene
utilisateur	1-N	avis	Avis rédigés (liés à une commande)

4.3 MCD (Merise)

Le schéma ci-dessous suit la convention Merise : chaque cardinalité est portée du côté de l'entité dont elle décrit la participation (ex. UTILISATEUR (1,1) possède ROLE (0,n) : un utilisateur possède exactement un rôle, un rôle peut être porté par plusieurs utilisateurs).

MCD - Vite & Gourmand (v2 - cardinalités corrigées)

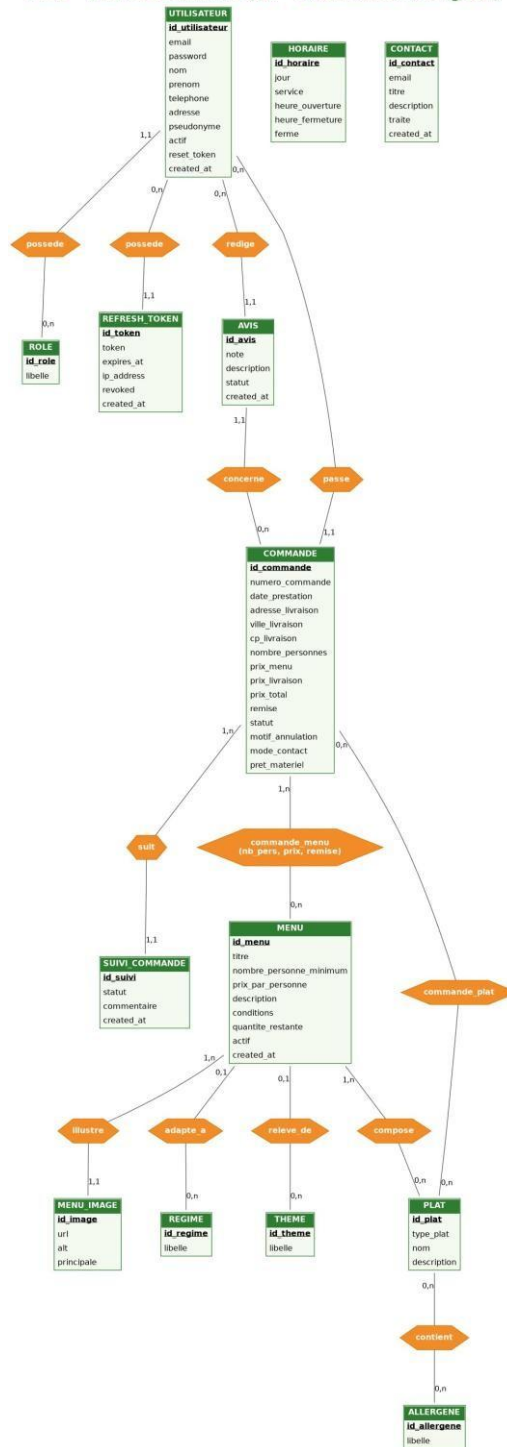


Figure 1 — Modèle Conceptuel de Données (v2)

4.4 Diagramme de cas d'utilisation

Quatre acteurs interagissent avec le système. L'héritage suit la hiérarchie des rôles : le client hérite des cas du visiteur, l'employé de ceux du client, l'administrateur de ceux de l'employé. Les relations «include» matérialisent l'authentification obligatoire, les relations «extend» les cas optionnels.

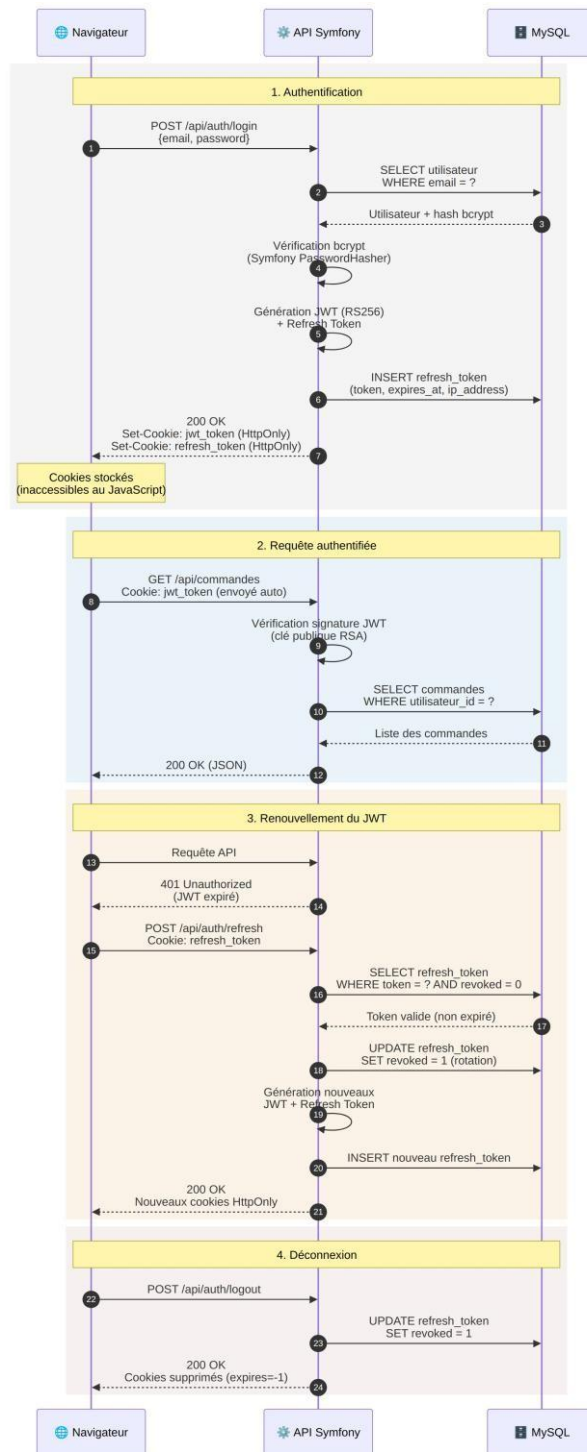


Figure 3 — Séquence authentification JWT (rotation des refresh tokens)

5. API REST

5.1 Authentification

L'API utilise JWT (RS256, clés RSA 4096 bits). Le token est stocké dans un cookie HttpOnly (non accessible au JavaScript, protection contre le vol par XSS), accompagné d'un refresh token à rotation. La connexion pose automatiquement les cookies.

Méthode	Route	Auth	Description
POST	/api/auth/register	Public	Créer un compte utilisateur
POST	/api/auth/login	Public	Connexion : pose les cookies JWT
POST	/api/auth/refresh	Cookie	Renouvelle le JWT via refresh token
POST	/api/auth/logout	Cookie	Déconnexion : supprime les cookies
GET	/api/auth/me	JWT	Profil de l'utilisateur connecté
PUT	/api/auth/me	JWT	Modifier son profil
DELETE	/api/auth/me	JWT	Supprimer son compte (RGPD)
POST	/api/auth/forgot-password	Public	Demande de réinitialisation
POST	/api/auth/reset-password	Public	Réinitialiser via token e-mail

5.2 Menus & Commandes

Méthode	Route	Auth	Description
GET	/api/menus	Public	Liste (filtres : thème, régime, prix, personnes)
GET	/api/menus/{id}	Public	Détail d'un menu (plats, images)
POST	/api/menus	Employé	Créer un menu
PUT	/api/menus/{id}	Employé	Modifier un menu
DELETE	/api/menus/{id}	Employé	Désactiver (soft delete)
GET	/api/commandes	JWT	Mes commandes
POST	/api/commandes	JWT	Créer une commande (multi-menus) : 400 si aucun menu demandé n'est disponible
PUT	/api/commandes/{id}	JWT	Modifier (si statut en_attente)
DELETE	/api/commandes/{id}	JWT	Annuler (si statut en_attente)
GET	/api/commandes/toutes	Employé	Toutes les commandes
PATCH	/api/commandes/{id}/statut	Employé	Changer le statut

Ajout v1.4 Validation de la disponibilité des menus. La création d'une commande vérifiait uniquement que le payload contenait au moins un menu. Or le service filtre ensuite chaque menu demandé sur trois critères : existence, statut actif, stock restant strictement positif, et ignore silencieusement ceux qui échouent. Lorsque tous les menus étaient filtrés, la commande était malgré tout persistée avec un prix de menu nul et aucun menu associé, et la requête se terminait en erreur 500 lors de la composition de l'e-mail de confirmation. Le contrôle porte désormais sur le résultat du filtrage et non sur le payload : si aucun menu retenu ne subsiste, une `InvalidArgumentException` est levée avant toute persistance, et le contrôleur la traduit en réponse 400 assortie d'un message explicite. Le calcul des frais de livraison, qui déclenche jusqu'à trois appels à l'API `OpenRouteService`, a été déplacé après ce contrôle afin de ne plus solliciter le service externe pour une commande vouée au rejet.

5.3 Administration & autres

Méthode	Route	Auth	Description
GET	/api/admin/employes	Admin	Liste des employés
POST	/api/admin/employes	Admin	Créer un compte employé
PATCH	/api/admin/employes/{id}/toggle	Admin	Activer / désactiver
GET	/api/admin/stats	Admin	Statistiques MongoDB (filtres menu, dates)
GET	/api/avis	Public	Avis validés publics
PATCH	/api/avis/{id}/valider	Employé	Valider un avis
PATCH	/api/avis/{id}/refuser	Employé	Refuser un avis
POST	/api/contact	Public	Envoyer un message de contact
GET	/api/doc	Public	Documentation Swagger UI

6. Sécurité

6.1 Authentification JWT

Aspect	Détail
Algorithme	RS256 : clés asymétriques RSA 4096 bits
Durée de vie	Token courte durée (1 h) + refresh token à rotation (30 j)
Stockage client	Cookie HttpOnly (inaccessible au JavaScript)
Protection XSS	Le token n'étant pas en localStorage, il ne peut être volé par script
Refresh token	Rotation à chaque renouvellement + révocation en base, cookie limité au chemin /api/auth
Cookies	SameSite=Lax + Secure (même domaine grâce au proxy Vercel)
Clés	private.pem / public.pem : hors Git, contenu en base64 dans les variables d'environnement

Choix d'architecture : le stockage du JWT en cookie HttpOnly (plutôt qu'en localStorage) protège contre l'exfiltration du token par injection de script (XSS). La contrainte cross-origin initiale entre le front (Vercel) et le back (Render) a été levée par la mise en place d'un proxy Vercel (voir 6.4) : le navigateur n'échange qu'avec un seul domaine.

6.2 Contrôle d'accès (RBAC)

Rôle	Périmètre
ROLE_USER	Utilisateur authentifié : commandes, profil, avis
ROLE_EMPLOYE	Hérite de USER : gestion menus, commandes, avis, horaires
ROLE_ADMIN	Hérite de EMPLOYE : gestion employés, statistiques

Le contrôle est appliqué à deux niveaux complémentaires : la liste `access_control` de `security.yaml`, évaluée avant l'entrée dans le contrôleur, et l'attribut `#[IsGranted('ROLE_X')]` posé sur les classes et les méthodes de contrôleur. La hiérarchie de rôles définie dans `security.yaml` évite d'énumérer les rôles hérités sur chaque route.

Le contrôle des droits est systématiquement effectué côté serveur. Côté client, la fonction `Auth.hasRole()` lit les rôles depuis le `localStorage` : elle sert uniquement à afficher ou masquer des éléments d'interface et ne constitue en aucun cas une mesure de sécurité. Un utilisateur qui modifierait cette valeur ferait apparaître des écrans d'administration, mais toutes les requêtes correspondantes seraient rejetées par l'API.

6.3 Règle de repli sur les routes API

Ajout v1.4. La liste `access_control` énumérait explicitement les préfixes de routes publiques, utilisateur, employé et administrateur, mais ne comportait aucune règle terminale. Symfony retenant la première règle correspondante et n'appliquant aucune restriction lorsqu'aucune ne correspond, toute route nouvellement ajoutée sous `/api` dont le préfixe n'aurait pas été déclaré serait devenue accessible sans authentification. Une règle de repli - `{ path: ^/api, roles: IS_AUTHENTICATED_FULLY }` a été ajoutée en dernière position : elle n'a aucun effet sur les routes déjà couvertes, puisqu'une règle antérieure les capte, et impose l'authentification à toute route non déclarée. Le principe appliqué est celui du refus par défaut : l'ouverture d'une route devient un acte explicite.

L'ordre des règles est signifiant et a été établi en conséquence : les routes publiques les plus spécifiques précèdent les règles générales qui les englobent. Ainsi `^/api/commandes/livraison` en accès public est déclaré avant `^/api/commandes` réservé à `ROLE_USER`, et la lecture publique de `^/api/avis` en GET précède l'écriture réservée aux utilisateurs authentifiés.

6.4 Cookies cross-origin et protection CSRF

Le front (Vercel) et le back (Render) étant initialement sur deux domaines distincts, les cookies devaient être émis en SameSite=None, une configuration fragilisée par le blocage des cookies tiers de Chrome 120+. Un proxy a donc été mis en place dans vercel.json : les requêtes /api/* émises vers le domaine du front sont réécrites côté serveur vers l'API Render. Du point de vue du navigateur, front et API partagent le même domaine, ce qui a permis de passer les cookies en SameSite=Lax + Secure, plus restrictif et plus sûr contre le CSRF.

La protection CSRF repose ainsi sur trois éléments combinés : l'attribut SameSite=Lax des cookies, la politique CORS restrictive (seule l'origine du front de production est autorisée pour les accès directs à l'API), et l'obligation d'envoyer Content-Type: application/json sur toutes les requêtes, un formulaire HTML externe ne peut pas forger une requête valide. L'API étant stateless (JWT), la protection CSRF classique par jeton de session ne s'applique pas.

6.5 En-têtes de sécurité HTTP

Ajout v1.4 Couverture du domaine front. Les en-têtes de sécurité et la Content Security Policy étaient configurés via NelmioSecurityBundle, côté Symfony. Or Symfony ne sert que l'API : la SPA est distribuée par Vercel, sur un domaine distinct qui ne traversait aucun code PHP. Le domaine effectivement visité par les utilisateurs, et sur lequel s'exécute l'intégralité du JavaScript de l'application, ne recevait donc aucun en-tête de sécurité : ni CSP, ni XFrame-Options, ni nosniff, ni HSTS. La CSP stricte configurée côté API ne s'appliquait pas là où le risque d'injection de script se matérialise. Un bloc headers a été ajouté à vercel.json pour couvrir l'ensemble des réponses du front.

En-tête	Valeur et objectif
Strict-Transport-Security	max-age=31536000 ; includeSubDomains : impose HTTPS au navigateur
Content-Security-Policy	default-src 'self' ; script-src limité à 'self' et au CDN jsDelivr ; object-src 'none' ; base-uri 'self' ; frame-ancestors 'none'
X-Content-Type-Options	nosniff : empêche la réinterprétation du type MIME
X-Frame-Options	SAMEORIGIN : protection contre le clickjacking
Referrer-Policy	strict-origin-when-cross-origin : limite la fuite d'URL
Permissions-Policy	caméra, micro, géolocalisation, paiement et USB désactivés

La CSP du front autorise le domaine cdn.jsdelivr.net pour les scripts et les styles, ainsi que fonts.googleapis.com et fonts.gstatic.com pour les polices : Bootstrap, ses icônes, Chart.js et les polices du projet sont chargés depuis ces CDN. Aucune source inline n'est autorisée pour les scripts. Les deux gestionnaires d'événements inline (attributs onclick) que comportait encore le code ont été remplacés par des attributs de données, data-nav et data-close-modal, traités par délégation d'événements sur document dans js/script.js. La délégation présente ici un avantage supplémentaire : elle fonctionne sur les éléments injectés dynamiquement par le routeur, ce qu'un écouteur posé au chargement ne permettrait pas.

La directive `style-src` conserve `'unsafe-inline'`, Bootstrap et Chart.js appliquant des styles calculés en ligne. Cette tolérance est limitée aux feuilles de style et ne s'étend pas aux scripts, où réside le risque d'exécution de code injecté.

Le résultat a été mesuré avec le service securityheaders.com : le domaine du front, précédemment noté F faute d'en-têtes, obtient la note A+ après déploiement. Côté API, l'en-tête HSTS est posé par Apache dans le Dockerfile et les autres en-têtes par NelmioSecurityBundle ; la CSP y est désactivée sur la seule route `/api/doc`, la page Swagger UI exécutant un script inline.

6.6 Résilience des services externes

Ajout v1.4. La création et la mise à jour d'une commande déclenchent, après l'enregistrement en base, deux traitements dépendant de services tiers : la synchronisation du document de statistiques dans MongoDB Atlas et l'envoi d'un e-mail transactionnel via Brevo. Ces appels n'étaient pas protégés : l'indisponibilité de l'un ou l'autre propageait une exception jusqu'au contrôleur et se traduisait par une erreur 500, alors même que la commande avait été correctement persistée. L'utilisateur constatait un échec pour une opération réussie, et pouvait la relancer, créant un doublon.

Ces deux traitements sont désormais encadrés par une gestion d'erreur dédiée : l'échec est journalisé via le `LoggerInterface` de Symfony, avec le numéro de commande et le message d'origine, mais n'interrompt pas la requête. La commande est confirmée à l'utilisateur, et l'incident reste traçable dans les journaux applicatifs.

Ce choix repose sur une distinction entre traitement critique et effet de bord. L'écriture en base relationnelle est critique : son échec doit faire échouer l'opération. La statistique MongoDB et l'e-mail de confirmation sont des effets de bord, leur perte est regrettable mais n'invalide pas la commande. Le même raisonnement était déjà appliqué au calcul des frais de livraison : en cas d'indisponibilité d'`OpenRouteService`, `DeliveryService` applique un forfait de repli plutôt que d'interrompre la commande.

Les quatre méthodes concernées de `CommandeService`, création, modification, annulation et changement de statut, ont été alignées sur ce principe. Dans la méthode de changement de statut, l'enregistrement en base a par ailleurs été déplacé avant les envois d'e-mail, afin que le nouveau statut soit persisté même si la notification échoue.

6.7 Intégrité du stock à l'annulation

Ajout v1.4. L'annulation d'une commande restituait une unité de stock au seul menu référencé par la clé étrangère menu de la commande. Ce champ ne conserve que le premier menu retenu lors de la création ; pour une commande multi-menus, les unités décrémentées sur les menus suivants n'étaient donc jamais restituées, produisant une érosion progressive et silencieuse des stocks. La restitution parcourt désormais la collection `commandeMenus`, qui reflète l'intégralité des menus effectivement commandés.

Cette correction met en lumière une double modélisation du lien entre commande et menu : une association simple, héritée de la version mono-menu de l'application, et une association porteuse d'attributs introduite pour le multi-menus. Les deux coexistent, la première servant d'accès rapide au menu principal. Cette redondance est identifiée comme dette technique et inscrite aux perspectives d'évolution ; la méthode de modification d'une commande s'appuie encore sur l'association simple et ne recalcule donc le prix que pour le menu principal.

6.8 Génération des références de commande

Ajout v1.4. Le numéro de commande était construit à partir de `uniqid()` tronqué à ses six derniers caractères. Cette fonction dérive sa valeur de l'horloge système : elle n'est pas cryptographiquement aléatoire, et la troncature réduisait encore l'espace des valeurs possibles. Sur une colonne portant une contrainte d'unicité, une collision se serait traduite par une erreur serveur au moment de l'enregistrement. La génération repose désormais sur `random_bytes(4)`, source d'aléa cryptographique, convertie en huit caractères hexadécimaux.

6.9 Autres mesures

Mesure de sécurité
Mots de passe hachés avec bcrypt (coût 12)
Politique de mot de passe centralisée dans une contrainte Symfony réutilisable (App\Validator\MotDePasseValide) : 10 caractères minimum, majuscule, minuscule, chiffre, caractère spécial
CORS restrictif : seule l'origine du front de production est autorisée
Secrets hors Git (.env.local) et injectés via variables d'environnement Render
Upload d'images : types MIME validés (JPG/PNG/WebP), taille limitée
Tokens de réinitialisation à usage unique et expiration courte
Rate limiting sur login / register / forgot-password
ExceptionHandler : messages d'erreur génériques en production (pas de fuite)
Routes {id} contraintes à \d+ : matching déterministe des routes littérales
Validation des statuts de commande contre la liste Commande::STATUTS avant traitement

La limitation du nombre de tentatives de connexion est assurée par un abonné d'événement dédié, LoginRateLimiterSubscriber, adossé au composant RateLimiter de Symfony. Il intercepte la requête avec une priorité supérieure à celle du pare-feu, donc avant toute vérification du mot de passe, applique un quota par adresse IP et retourne une réponse 429 accompagnée de l'en-tête Retry-After. Ce mécanisme couvrant déjà le besoin, l'option native login_throttling n'est pas activée : deux protections concurrentes sur la même route rendraient le comportement difficile à prévoir.

Ajout v1.4. La règle de robustesse des mots de passe était appliquée par une expression régulière dupliquée à trois endroits du contrôleur d'authentification : inscription, réinitialisation et modification du profil. Cette duplication exposait à une divergence entre les trois copies, et la règle n'était testable qu'indirectement, à travers des tests fonctionnels sollicitant l'API. Elle est désormais extraite dans une contrainte de validation Symfony réutilisable, App\Validator\MotDePasseValide, accompagnée de son validateur. Les trois points d'entrée délèguent au composant Validator, ce qui garantit un message et un comportement identiques, et la règle est couverte par huit tests unitaires : trois mots de passe valides et un cas par critère manquant. Une contrainte plutôt qu'une méthode privée, car elle reste applicable ailleurs, par annotation sur une entité par exemple, et se teste en isolation, sans base de données ni requête HTTP.

7. Déploiement

7.1 Frontend Vercel

Paramètre	Valeur
-----------	--------

Plateforme	Vercel : hébergement statique gratuit
Repository	github.com/NicolasVrignaudVG/Vite-Gourmand
Branche	main : déploiement auto à chaque push (webhook)
Build	Aucun build : HTML/CSS/JS servis directement
Routing SPA	vercel.json : proxy /api/* vers Render, puis catch-all vers index.html
Proxy API	Rewrites Vercel : le front appelle /api/* sur son propre domaine ; Vercel relaie vers l'API Render (cookies same-site)
En-têtes de sécurité	Bloc headers de vercel.json : appliqué à toutes les réponses du front (voir §6.5)

Le fichier vercel.json remplit désormais deux fonctions distinctes : le bloc rewrites assure le proxy vers l'API et le routage de la SPA, le bloc headers applique les en-têtes de sécurité. Les deux sont indépendants : un rewrite ne transmet pas les en-têtes du bloc headers vers le back, et réciproquement les réponses proxifiées conservent les en-têtes émis par Symfony.

7.2 Backend Render (Docker)

Paramètre	Valeur
Plateforme	Render : Web Service Docker
Repository	github.com/NicolasVrignaudVG/Vite-Gourmand-back
Image	php:8.4-apache + extensions mongodb, pdo_mysql, intl
Port	10000 (attendu par le scan de ports Render)
Démarrage	assets:install, asset-map:compile, puis apache2-foreground
En-têtes	HSTS posé par Apache (a2enmod headers) ; autres en-têtes via NelmioSecurityBundle
Limites	Fichiers éphémères : images perdues au redéploiement

7.3 Bases de données

Service	Détail
MySQL (Clever Cloud)	MySQL 8.0 managé : charset utf8mb4, 5 connexions max (gratuit)
Migrations	Doctrine Migrations appliquées au déploiement
MongoDB Atlas	Cluster M0 gratuit : base de statistiques

Collection stats	commandes : upsert à chaque commande (CA, volumes par menu)
------------------	---

7.4 Variables d'environnement (Render)

Variable	Description
APP_ENV	prod
APP_SECRET	Clé secrète Symfony
DATABASE_URL	URL MySQL Clever Cloud (identifiants masqués)
MONGO_URI / MONGO_DB	Connexion MongoDB Atlas + nom de base
MAILER_DSN	brevo+api://KEY@default
MAILER_SENDER_EMAIL / MAILER_SENDER_NAME	Adresse et nom d'expéditeur (masqués)
CORS_ALLOW_ORIGIN	Origine du front autorisée
JWT_SECRET_KEY / JWT_PUBLIC_KEY	Contenu des clés RSA encodé en base64
JWT_PASSPHRASE	Passphrase de la clé privée
ORS_API_KEY	Clé API OpenRouteService

Aucune valeur réelle de secret n'est reproduite dans ce document : identifiants de base, adresses e-mail et clés sont volontairement masqués. Les variables `JWT_SECRET_KEY` et `JWT_PUBLIC_KEY` contiennent le contenu des fichiers de clés encodé en base64, et non un chemin d'accès, la configuration de `LexikJWTAuthenticationBundle` utilisant le processeur `%env(base64:...)%`, un chemin y serait interprété comme une donnée à décoder et provoquerait un échec de signature des tokens.

8. Structure du code & gestion de versions

8.1 Backend Symfony

Dossier	Rôle
src/Controller/	Contrôleurs API (Auth, Commande, Menu, Admin...)
src/Entity/	Entités Doctrine
src/Repository/	Composants d'accès aux données : requêtes personnalisées construites avec le QueryBuilder de Doctrine
src/Service/	Services métier (CommandeService, MenuService, MailerService, MongoService, DeliveryService)
src/EventListener/	ExceptionListener : gestion JSON des erreurs
src/EventSubscriber/	LoginRateLimiterSubscriber : limitation des tentatives de connexion

src/Validator/	Contraintes de validation réutilisables (MotDePasseValide)
src/Security/	JwtCookieSuccessHandler : émission des cookies JWT et refresh
migrations/	Migrations Doctrine
config/jwt/	Clés RSA (non committées)
Dockerfile	Image PHP 8.4 Apache pour Render

Ajout v1.4 Optimisation de l'accès aux données. La récupération des commandes d'un utilisateur s'appuyait sur la méthode générique `findBy()` de Doctrine. Chaque commande sérialisée traverse ensuite son menu, son utilisateur, ses suivis, ses menus commandés et ses plats choisis ; ces associations étant chargées à la demande, une requête supplémentaire était émise à chaque traversée, comportement connu sous le nom de problème N+1, dont le coût croît linéairement avec le nombre de commandes affichées. Une méthode `findByUtilisateurAvecDetails()` a été ajoutée au `CommandeRepository` : construite avec le `QueryBuilder`, elle déclare les jointures nécessaires et les inclut à la sélection via `addSelect()`, de sorte que l'ensemble du graphe d'objets est hydraté par une requête unique. Le paramètre utilisateur est lié plutôt que concaténé, conformément à la protection contre l'injection SQL assurée par Doctrine.

Ajout v1.4. La création et la mise à jour d'un menu s'appuyaient sur une méthode privée du contrôleur, `hydrateMenu`, qui portait la logique d'association du thème et du régime (avec création de l'enregistrement s'il n'existe pas encore) ainsi que la gestion de l'image principale, remplacée si elle existe déjà, créée sinon. Cette logique est métier et non liée au transport HTTP : elle a été extraite dans un `MenuService`, sur le modèle du `CommandeService` introduit précédemment pour corriger le même anti-pattern du « Fat Controller ». Le contrôleur se limite désormais à décoder la requête, déléguer au service et formater la réponse. La logique extraite est découpée en trois méthodes explicites, `associerTheme`, `associerRegime` et `associerImagePrincipale`, et devient testable en isolation, sans requête HTTP ni authentification.

8.2 Frontend JavaScript

Fichier	Rôle
js/api.js	Fonctions d'appel API (Auth, Menus, Commandes, Admin)
js/script.js	Logique des pages, gestion d'état, rendu, et délégation d'événements pour les attributs <code>data-nav</code> et <code>data-close-modal</code>
Router/router.js	Routeur SPA : chargement dynamique des fragments (module ES, dossier avec majuscule)
css/main.css	Design tokens, composants, mise en page
pages/*.html	Fragments HTML des pages
index.html	Point d'entrée SPA
Fichier	Rôle

*Correction v1.4 Casse du chemin du routeur. Le fichier était référencé dans cette documentation sous le chemin `js/router.js` ; il se trouve en réalité dans un dossier `Router/` à la racine, avec une majuscule initiale, et est importé comme module ES depuis `index.html`. La distinction n'est pas cosmétique : les systèmes de fichiers de Windows sont insensibles à la casse, ceux des plateformes de déploiement ne le sont pas. Un écart de casse entre le code et le dépôt passe donc inaperçu en développement et se traduit par une erreur 404 en production. **Ajout v1.4.** Les options des filtres thème et régime de la page Menus étaient déclarées en dur dans le HTML. Un thème créé depuis l'espace employé était donc correctement enregistré et associé au menu, mais n'apparaissait jamais dans le filtre public, qui restait figé sur les valeurs du jeu de données initial. La fonction `remplirSelectFiltre` construit désormais ces options à partir des valeurs distinctes présentes dans les menus déjà chargés : aucune requête supplémentaire n'est nécessaire, la réponse de `GET /api/menus` portant déjà le thème et le régime de chaque menu. Un effet secondaire souhaitable est qu'un thème auquel aucun menu actif n'est rattaché n'apparaît plus dans le filtre, évitant une option sans résultat possible.*

8.3 Stratégie de branches Git

Le projet suit un modèle à trois niveaux : main (production, déployée), develop (intégration), et des branches `feature/*` par fonctionnalité, mergées vers develop après test puis vers main.

Branche	Contenu
main	Code de production : déploiement automatique
develop	Intégration des fonctionnalités
feature/authentification	Connexion, inscription, JWT, reset password
feature/gestion-menus	CRUD menus, plats, images, thèmes, régimes
feature/commandes	Commande multi-menus, calcul livraison
feature/espace-utilisateur	Espace client : historique, profil
feature/espace-employe	Gestion commandes, avis, horaires
feature/espace-admin	Gestion employés, statistiques MongoDB

9. Dette technique identifiée

Section ajoutée en v1.4.

Les points ci-dessous sont connus et documentés. Ils n'affectent pas le fonctionnement de l'application dans son périmètre actuel, mais mériteraient un traitement dans une évolution ultérieure.

Point	Analyse
-------	---------

Type des montants	Les colonnes tarifaires sont en DOUBLE ; DECIMAL serait préférable pour des valeurs monétaires. Migration à prévoir sur les entités, les migrations et la sérialisation.
Double modélisation menu	La commande porte à la fois une association simple vers un menu et une association porteuse d'attributs. La modification d'une commande ne recalcule le prix que du menu principal.
Route de calcul de livraison	La route /api/commandes/livraison est déclarée publique dans access_control mais appartient à un contrôleur porteur d'un IsGranted de classe. Sans effet aujourd'hui, le front ne l'appelant pas.
Concurrence sur le stock	La décrémentation du stock n'est pas verrouillée : deux commandes simultanées sur la dernière unité disponible pourraient être acceptées.